Lab: Deploy a Fully Private Azure Databricks Workspace with Private Link using Terraform

Description

In this lab, you will extend the VNet injection lab to build a **fully private Azure Databricks workspace** using **Azure Private Link** and Terraform (AzureRM v5.6.0).

You will add:

- A **back-end private endpoint**, so clusters reach the Databricks control plane over a private IP.
- A **front-end private endpoint**, so users reach the workspace UI and API over a private IP.
- **Private DNS zones** that make the workspace URL resolve to those private IPs.

Finally, you will **disable public network access**, so the workspace can only be reached from your private networks.

Private endpoints are easy to create. Getting **DNS** right is the hard part, and it is where most private deployments break. You will verify DNS from inside and outside the network, then deliberately break it to see what failure looks like.

Learning Objectives

By the end of this lab, you will be able to:

1. Explain the difference between **front-end** and **back-end** Private Link for Azure Databricks.
2. Create a dedicated **private endpoint subnet** in the workspace VNet.
3. Deploy a workspace with **public_network_access_enabled = false** and **network_security_group_rules_required = "NoAzureDatabricksRules"**.
4. Create **private endpoints** for the **databricks_ui_api** and **browser_authentication** sub-resources.
5. Create **Private DNS zones** (privatelink.azuredatabricks.net) and link them to the right VNets.

6. Verify name resolution and connectivity from a **jump VM** inside the private network.
7. Diagnose a **broken DNS** configuration, the most common Private Link failure.

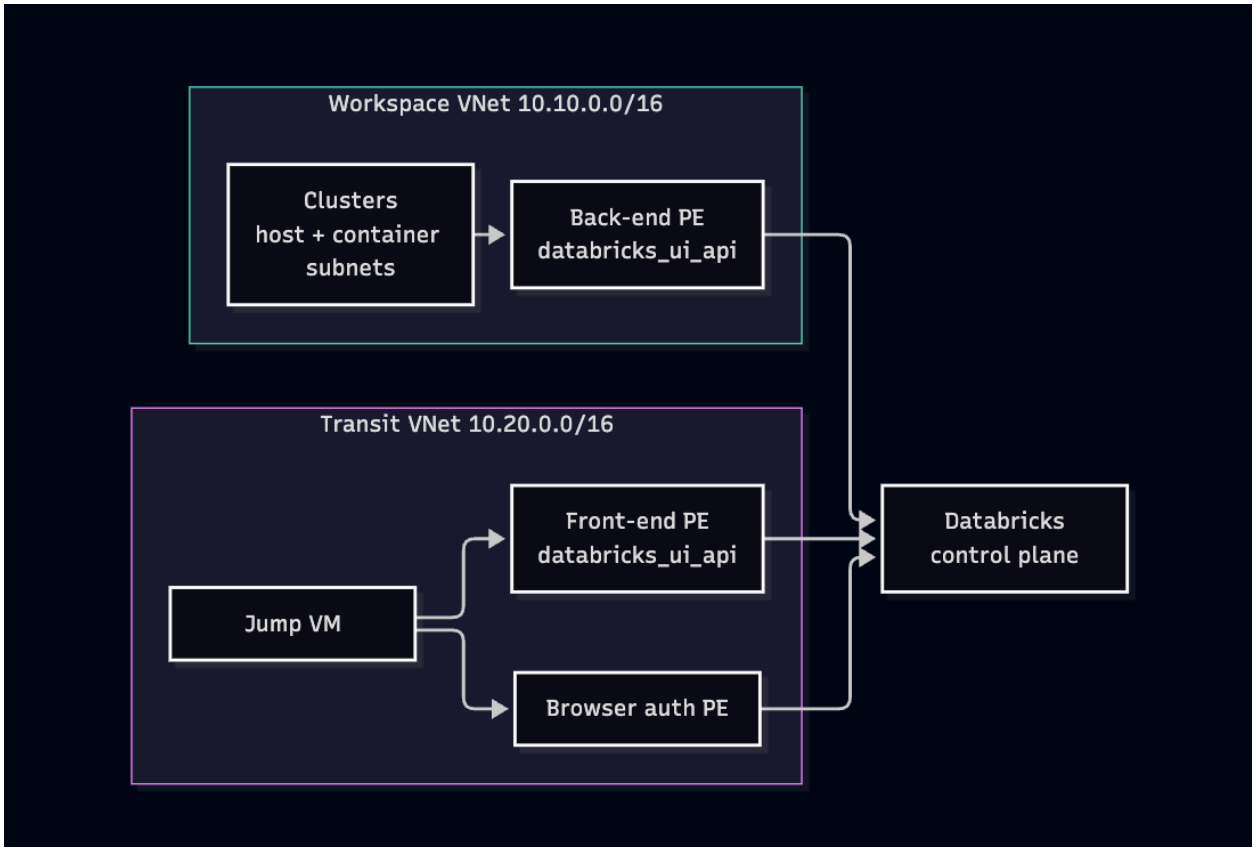
Prerequisites

Make sure you have the following before starting:

- A valid **Azure subscription** where you have **Contributor** (or Owner) access.
- **Terraform** and **Azure CLI** installed locally.
- **Visual Studio Code (VS Code)** or any IDE, and an **SSH client**.
- Completion of the previous lab, *Deploy Azure Databricks with VNet Injection using Terraform*.

*Private Link requires the **Premium** pricing tier.*

Architecture



Users connect from the transit VNet through the front-end endpoint. Clusters in the workspace VNet connect through the back-end endpoint. Public access is disabled, so nothing else can reach the workspace.

How DNS Resolution Works

The workspace URL (for example adb-1234567890.12.azuredatabricks.net) is public. Azure DNS answers with a CNAME to adb-1234567890.12.privatelink.azuredatabricks.net. What happens next depends on where the query comes from:

Where the lookup happens	Private DNS zone linked?	URL resolves to	Result
Your laptop (internet)	No	Public IP	Blocked: public access disabled
Jump VM (transit VNet)	Yes, transit zone	Front-end PE private IP	Allowed
Clusters (workspace VNet)	Yes, workspace zone	Back-end PE private IP	Allowed

Each VNet links to its own copy of the privatelink.azuredatabricks.net zone, so the same URL resolves to a different private IP in each network. The two zones have the same name, so they live in separate resource groups.

If a VNet has no zone linked, the URL silently resolves to the public IP and the connection is refused. This is the most common Private Link failure.

Step by Step Guide

Step 1: Authenticate and Find Your Public IP

Log in and select your subscription:

```
az login
az account set --subscription "<your-subscription-id>"
```

Find your public IP address. You will use it to allow SSH to the jump VM from your machine only:

```
curl -s https://ifconfig.me
```

Step 2: Create the Project and Configure the Providers

Create a new folder called databricks_private_link. Inside it, create a file named providers.tf:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "5.6.0"
    }

    tls = {
      source  = "hashicorp/tls"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = "<your-subscription-id>"

  resource_providers_to_register = [
    "Microsoft.Databricks",
    "Microsoft.Network",
    "Microsoft.Compute",
  ]
}
```

Explanation:

- tls → generates the SSH key for the jump VM.
- Microsoft.Compute → needed for the jump VM.

Initialize the project:

```
terraform init
```

Step 3: Define Variables and Values

Create a file named variables.tf:

```
# ---------- General ----------
variable "location" {
  type    = string
  default = "West Europe"
}

variable "tags" {
  type = map(string)
}

# ---------- Workspace network ----------
variable "resource_group_name" {
  type = string
}

variable "vnet_name" {
  type = string
}

variable "vnet_address_space" {
  type    = list(string)
  default = ["10.10.0.0/16"]
}

variable "public_subnet_prefix" {
  type    = list(string)
  default = ["10.10.1.0/24"]
}

variable "private_subnet_prefix" {
  type    = list(string)
  default = ["10.10.2.0/24"]
}
```

```
variable "backend_pe_subnet_prefix" {
  type    = list(string)
  default = ["10.10.3.0/27"]
}

# ---------- Workspace ----------
variable "workspace_name" {
  type = string
}

variable "managed_resource_group_name" {
  type = string
}

# ---------- Transit network ----------
variable "transit_resource_group_name" {
  type = string
}

variable "transit_vnet_name" {
  type = string
}

variable "transit_vnet_address_space" {
  type    = list(string)
  default = ["10.20.0.0/16"]
}

variable "frontend_pe_subnet_prefix" {
  type    = list(string)
  default = ["10.20.1.0/27"]
}

variable "jumpbox_subnet_prefix" {
  type    = list(string)
```

```
  default = ["10.20.2.0/27"]
}

variable "jumpbox_vm_name" {
  type = string
}

variable "my_ip_address" {
  type        = string
  description = "Your public IP in CIDR form, e.g. 203.0.113.10/32"
}
```

Create a file named terraform.tfvars. Replace the IP with the one from Step 1:

```
location                    = "West Europe"
resource_group_name         = "rg-databricks-pl-lab"
vnet_name                   = "vnet-databricks-pl"
workspace_name              = "dbw-private-lab"
managed_resource_group_name = "rg-databricks-pl-lab-managed"
transit_resource_group_name = "rg-databricks-transit-lab"
transit_vnet_name           = "vnet-transit-pl"
jumpbox_vm_name             = "vm-jumpbox"
my_ip_address               = "203.0.113.10/32"
tags = {
  environment = "dev"
  project     = "databricks-private-link-lab"
}
```

Step 4: Create the Resource Groups

You need two resource groups: one for the workspace network and one for the transit network. Each holds its own copy of the private DNS zone.

Create a file named resource_groups.tf:

```
resource "azurerm_resource_group" "workspace" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
```

```
}

resource "azurerm_resource_group" "transit" {
  name     = var.transit_resource_group_name
  location = var.location
  tags     = var.tags
}
```

Step 5: Create the Workspace VNet and Subnets

This time the VNet has three subnets: the two Databricks subnets from the previous lab, plus a dedicated subnet for the back-end private endpoint.

Create a file named workspace_network.tf:

```
locals {
  databricks_subnets = {
    host      = { name = "snet-databricks-host", prefix = var.public_subnet_prefix }
    container = { name = "snet-databricks-container", prefix = var.private_subnet_prefix }
  }
}

resource "azurerm_virtual_network" "workspace" {
  name                = var.vnet_name
  address_space       = var.vnet_address_space
  location            = azurerm_resource_group.workspace.location
  resource_group_name = azurerm_resource_group.workspace.name
  tags                = var.tags
}

resource "azurerm_subnet" "databricks" {
  for_each = local.databricks_subnets

  name                 = each.value.name
  resource_group_name  = azurerm_resource_group.workspace.name
  virtual_network_name = azurerm_virtual_network.workspace.name
  address_prefixes     = each.value.prefix
```

```
  delegation {
    name = "databricks-${each.key}-delegation"

    service_delegation {
      name = "Microsoft.Databricks/workspaces"
      actions = [
        "Microsoft.Network/virtualNetworks/subnets/join/action",
        "Microsoft.Network/virtualNetworks/subnets/prepareNetworkPolicies/action",
        "Microsoft.Network/virtualNetworks/subnets/unprepareNetworkPolicies/action",
      ]
    }
  }
}

resource "azurerm_subnet" "backend_pe" {
  name                 = "snet-backend-pe"
  resource_group_name  = azurerm_resource_group.workspace.name
  virtual_network_name = azurerm_virtual_network.workspace.name
  address_prefixes     = var.backend_pe_subnet_prefix
}
```

Explanation:

- for_each → creates both Databricks subnets from one block, instead of repeating the code.
- snet-backend-pe → holds only the private endpoint. It is not delegated to Databricks.

Step 6: Create the NSG and Associate It

Create a file named nsg.tf:

```
resource "azurerm_network_security_group" "workspace" {
  name                = "nsg-databricks-pl"
  location            = azurerm_resource_group.workspace.location
  resource_group_name = azurerm_resource_group.workspace.name
  tags                = var.tags
}
```

```
resource "azurerm_subnet_network_security_group_association" "databricks" {
  for_each = azurerm_subnet.databricks

  subnet_id                 = each.value.id
  network_security_group_id = azurerm_network_security_group.workspace.id
}
```

Step 7: Create the NAT Gateway

Clusters still need outbound access for libraries and package repositories. Create a file named nat_gateway.tf:

```
resource "azurerm_public_ip" "nat" {
  name                = "pip-nat-databricks-pl"
  location            = azurerm_resource_group.workspace.location
  resource_group_name = azurerm_resource_group.workspace.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = var.tags
}

resource "azurerm_nat_gateway" "workspace" {
  name                    = "nat-databricks-pl"
  location                = azurerm_resource_group.workspace.location
  resource_group_name     = azurerm_resource_group.workspace.name
  sku_name                = "Standard"
  idle_timeout_in_minutes = 10
  tags                    = var.tags
}

resource "azurerm_nat_gateway_public_ip_association" "workspace" {
  nat_gateway_id       = azurerm_nat_gateway.workspace.id
  public_ip_address_id = azurerm_public_ip.nat.id
}

resource "azurerm_subnet_nat_gateway_association" "databricks" {
  for_each = azurerm_subnet.databricks
```

```
  subnet_id      = each.value.id
  nat_gateway_id = azurerm_nat_gateway.workspace.id
}
```

Step 8: Create the Private Workspace

Create a file named databricks_workspace.tf:

```
resource "azurerm_databricks_workspace" "workspace" {
  name                        = var.workspace_name
  resource_group_name         = azurerm_resource_group.workspace.name
  location                    = azurerm_resource_group.workspace.location
  sku                         = "premium"
  managed_resource_group_name = var.managed_resource_group_name

  public_network_access_enabled         = false
  network_security_group_rules_required = "NoAzureDatabricksRules"

  custom_parameters {
    no_public_ip        = true
    virtual_network_id  = azurerm_virtual_network.workspace.id
    public_subnet_name  = azurerm_subnet.databricks["host"].name
    private_subnet_name = azurerm_subnet.databricks["container"].name

    public_subnet_network_security_group_association_id  =
azurerm_subnet_network_security_group_association.databricks["host"].id
    private_subnet_network_security_group_association_id =
azurerm_subnet_network_security_group_association.databricks["container"].id
  }

  tags = var.tags

  depends_on = [azurerm_subnet_nat_gateway_association.databricks]
}
```

Explanation:

- sku = "premium" → Private Link is only available on the Premium tier.

- public_network_access_enabled = false → blocks all access from the internet. Only private endpoints can reach the workspace.
- NoAzureDatabricksRules → Databricks does not add NSG rules for control-plane traffic, because that traffic now flows through the back-end private endpoint.

Step 9: Create the Back-end Private DNS Zone

This zone lets clusters in the workspace VNet resolve the workspace URL to the back-end private endpoint.

Create a file named dns_backend.tf:

```
resource "azurerm_private_dns_zone" "backend" {
  name                = "privatelink.azuredatabricks.net"
  resource_group_name = azurerm_resource_group.workspace.name
  tags                = var.tags
}

resource "azurerm_private_dns_zone_virtual_network_link" "backend" {
  name                  = "link-workspace-vnet"
  private_dns_zone_id   = azurerm_private_dns_zone.backend.id
  virtual_network_id    = azurerm_virtual_network.workspace.id
  registration_enabled = false
  tags                  = var.tags
}
```

Explanation:

- Zone name → must be exactly privatelink.azuredatabricks.net. Any other name will not be used.
- VNet link → a zone only affects VNets it is linked to. Without this link, clusters resolve the public IP.
- private_dns_zone_id → from AzureRM v5.0, the link references the zone by ID instead of name and resource group.

Step 10: Create the Back-end Private Endpoint

Create a file named pe_backend.tf:

```
resource "azurerm_private_endpoint" "backend" {
 name                = "pe-databricks-backend"
 location            = azurerm_resource_group.workspace.location
 resource_group_name = azurerm_resource_group.workspace.name
 subnet_id           = azurerm_subnet.backend_pe.id

 private_service_connection {
  name                           = "psc-databricks-backend"
  private_connection_resource_id = azurerm_databricks_workspace.workspace.id
  is_manual_connection           = false
  subresource_names              = ["databricks_ui_api"]
 }

 private_dns_zone_group {
  name                 = "backend-dns"
  private_dns_zone_ids = [azurerm_private_dns_zone.backend.id]
 }

 tags = var.tags
}
```

Explanation:

- databricks_ui_api → the sub-resource for the workspace's control plane (UI and REST API). The back-end and front-end endpoints both use it.
- private_dns_zone_group → automatically creates the A record for the workspace in the zone. Without it, you would add records by hand.

Step 11: Create the Transit VNet

The transit VNet represents the network your users connect from. In production, this would be reached over VPN or ExpressRoute. In this lab, a jump VM stands in for a user's machine.

Create a file named transit_network.tf:

```
resource "azurerm_virtual_network" "transit" {
 name                = var.transit_vnet_name
 address_space       = var.transit_vnet_address_space
 location            = azurerm_resource_group.transit.location
```

```
  resource_group_name = azurerm_resource_group.transit.name
  tags                = var.tags
}

resource "azurerm_subnet" "frontend_pe" {
  name                 = "snet-frontend-pe"
  resource_group_name  = azurerm_resource_group.transit.name
  virtual_network_name = azurerm_virtual_network.transit.name
  address_prefixes     = var.frontend_pe_subnet_prefix
}

resource "azurerm_subnet" "jumpbox" {
  name                 = "snet-jumpbox"
  resource_group_name  = azurerm_resource_group.transit.name
  virtual_network_name = azurerm_virtual_network.transit.name
  address_prefixes     = var.jumpbox_subnet_prefix
}

resource "azurerm_network_security_group" "jumpbox" {
  name                = "nsg-jumpbox"
  location            = azurerm_resource_group.transit.location
  resource_group_name = azurerm_resource_group.transit.name

  security_rule {
    name                       = "allow-ssh-from-my-ip"
    priority                   = 1001
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = var.my_ip_address
    destination_address_prefix = "*"
  }

  tags = var.tags
```

```
}

resource "azurerm_subnet_network_security_group_association" "jumpbox" {
  subnet_id                 = azurerm_subnet.jumpbox.id
  network_security_group_id = azurerm_network_security_group.jumpbox.id
}
```

SSH is allowed only from your own IP address.

Step 12: Create the Front-end Private DNS Zone

This is a **second** zone with the **same name**, in the transit resource group, linked to the transit VNet.

Create a file named **dns_frontend.tf**:

```
resource "azurerm_private_dns_zone" "frontend" {
  name                = "privatelink.azuredatabricks.net"
  resource_group_name = azurerm_resource_group.transit.name
  tags                = var.tags
}

resource "azurerm_private_dns_zone_virtual_network_link" "frontend" {
  name                  = "link-transit-vnet"
  private_dns_zone_id   = azurerm_private_dns_zone.frontend.id
  virtual_network_id    = azurerm_virtual_network.transit.id
  registration_enabled = false
  tags                  = var.tags
}
```

Step 13: Create the Front-end and Browser Authentication Private Endpoints

Create a file named **pe_frontend.tf**:

```
resource "azurerm_private_endpoint" "frontend" {
  name                = "pe-databricks-frontend"
  location            = azurerm_resource_group.transit.location
  resource_group_name = azurerm_resource_group.transit.name
  subnet_id           = azurerm_subnet.frontend_pe.id
```

```
  private_service_connection {
    name                           = "psc-databricks-frontend"
    private_connection_resource_id = azurerm_databricks_workspace.workspace.id
    is_manual_connection           = false
    subresource_names              = ["databricks_ui_api"]
  }

  private_dns_zone_group {
    name                 = "frontend-dns"
    private_dns_zone_ids = [azurerm_private_dns_zone.frontend.id]
  }

  tags = var.tags
}

resource "azurerm_private_endpoint" "browser_auth" {
  name                = "pe-databricks-browser-auth"
  location            = azurerm_resource_group.transit.location
  resource_group_name = azurerm_resource_group.transit.name
  subnet_id           = azurerm_subnet.frontend_pe.id

  private_service_connection {
    name                           = "psc-databricks-browser-auth"
    private_connection_resource_id = azurerm_databricks_workspace.workspace.id
    is_manual_connection           = false
    subresource_names              = ["browser_authentication"]
  }

  private_dns_zone_group {
    name                 = "browser-auth-dns"
    private_dns_zone_ids = [azurerm_private_dns_zone.frontend.id]
  }

  tags = var.tags
```

```
  depends_on = [azurerm_private_endpoint.frontend]
}
```

Explanation:

- Front-end endpoint → gives users a private path to the workspace UI and REST API.
- browser_authentication → handles the Microsoft Entra ID single sign-on redirect when logging in through a browser. You need only one per region for each DNS zone.

In production, the browser authentication endpoint is usually attached to a small dedicated "web auth" workspace. That way, deleting a regular workspace does not break login for the others.

Step 14: Create the Jump VM

Create a file named jumpbox.tf:

```
resource "tls_private_key" "jumpbox" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

resource "azurerm_public_ip" "jumpbox" {
  name                = "pip-jumpbox"
  location            = azurerm_resource_group.transit.location
  resource_group_name = azurerm_resource_group.transit.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = var.tags
}

resource "azurerm_network_interface" "jumpbox" {
  name                = "nic-jumpbox"
  location            = azurerm_resource_group.transit.location
  resource_group_name = azurerm_resource_group.transit.name

  ip_configuration {
    name                          = "ipconfig-jumpbox"
    subnet_id                     = azurerm_subnet.jumpbox.id
```

```
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.jumpbox.id
  }

  tags = var.tags
}

resource "azurerm_linux_virtual_machine" "jumpbox" {
  name                            = var.jumpbox_vm_name
  location                        = azurerm_resource_group.transit.location
  resource_group_name             = azurerm_resource_group.transit.name
  size                            = "Standard_B2s"
  admin_username                  = "azureuser"
  disable_password_authentication = true
  network_interface_ids           = [azurerm_network_interface.jumpbox.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = tls_private_key.jumpbox.public_key_openssh
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "ubuntu-24_04-lts"
    sku       = "server"
    version   = "latest"
  }

  tags = var.tags
}
```

Step 15: Define Outputs

Create a file named outputs.tf:

```
output "workspace_hostname" {
  value = azurerm_databricks_workspace.workspace.workspace_url
}

output "backend_pe_private_ip" {
  value = azurerm_private_endpoint.backend.private_service_connection[0].private_ip_address
}

output "frontend_pe_private_ip" {
  value = azurerm_private_endpoint.frontend.private_service_connection[0].private_ip_address
}

output "jumpbox_public_ip" {
  value = azurerm_public_ip.jumpbox.ip_address
}

output "jumpbox_private_key" {
  value     = tls_private_key.jumpbox.private_key_pem
  sensitive = true
}
```

Step 16: Deploy the Infrastructure

```
terraform fmt
terraform validate
terraform plan
terraform apply --auto-approve
```

Deployment takes about 10 to 15 minutes. Private endpoints are created after the workspace.

View the outputs:

```
terraform output
```

Note the workspace_hostname, frontend_pe_private_ip and backend_pe_private_ip values.

Step 17: Verify the Private Setup

17.1 Check the Azure Portal

1. Open **dbw-private-lab** and go to **Settings → Networking**. **Public network access** shows **Disabled**.
2. Open the **Private endpoint connections** tab. You see **three** connections, all **Approved**.
3. Open the private DNS zone in **rg-databricks-pl-lab**. It has an **A record** for the workspace pointing to the back-end endpoint IP.
4. Open the private DNS zone in **rg-databricks-transit-lab**. It has an **A record** for the workspace pointing to the front-end endpoint IP, plus a **pl-auth** record for browser authentication.

17.2 Test from your laptop (outside the network)

```
nslookup $(terraform output -raw workspace_hostname)
```

The answer shows a CNAME to **privatelink.azuredatabricks.net** and a **public IP**. Open the workspace URL in your browser. Access is denied because public network access is disabled.

17.3 Test from the jump VM (inside the network)

Save the SSH key and connect:

```
terraform output -raw jumpbox_private_key > jumpbox_key.pem
chmod 400 jumpbox_key.pem
ssh -i jumpbox_key.pem azureuser@$(terraform output -raw jumpbox_public_ip)
```

On the jump VM, replace the hostname with your **workspace_hostname** output and run:

```
nslookup adb-1234567890.12.azuredatabricks.net
curl -s -o /dev/null -w "%{http_code}\n" https://adb-1234567890.12.azuredatabricks.net/
```

The name now resolves to the **front-end private IP** (10.20.1.x), matching **frontend_pe_private_ip**. curl returns an HTTP status code instead of a connection error, which confirms the workspace is reachable privately.

Step 18: Break DNS on Purpose

Now see what the most common Private Link failure looks like.

1. In dns_frontend.tf, comment out the azurerm_private_dns_zone_virtual_network_link.frontend block by adding # to the start of each line.
2. Apply the change. Terraform removes only the VNet link:

```
terraform apply --auto-approve
```

3. On the jump VM, clear the DNS cache and test again:

```
sudo resolvectl flush-caches
nslookup adb-1234567890.12.azuredatabricks.net
curl -s -o /dev/null -w "%{http_code}\n" https://adb-1234567890.12.azuredatabricks.net/
```

The name now resolves to the public IP, and access is refused.

What this teaches: the private endpoint still exists and still shows Approved in the portal. Everything looks healthy, but traffic never reaches the endpoint because DNS points elsewhere. When a private deployment fails, check DNS first.

4. Remove the # characters, run terraform apply --auto-approve again, flush the cache, and confirm the name resolves to the private IP once more.

Step 19: Clean Up Resources

Exit the jump VM, then destroy everything:

```
terraform destroy --auto-approve
```

This removes both resource groups and all resources in them. Azure deletes the managed resource group automatically. Delete jumpbox_key.pem from your machine.

Troubleshooting

- nslookup returns a public IP inside the VNet → the zone is not linked to that VNet, the zone name is wrong, or the result is cached. Check the link, then run sudo resolvectl flush-caches.
- The DNS zone has no A record → the private endpoint is missing its private_dns_zone_group block.
- Error about NSG rules on apply → confirm network_security_group_rules_required is NoAzureDatabricksRules and the back-end private endpoint is part of the configuration.

- Browser login fails with a redirect error → the browser_authentication endpoint or its pl-auth DNS record is missing.
- SSH times out → your public IP may have changed. Update my_ip_address and apply again.

Outcomes

You've deployed a fully private Azure Databricks workspace with Private Link using Terraform. You now know how to:

- Separate front-end (user) and back-end (cluster) traffic with dedicated private endpoints.
- Create the databricks_ui_api and browser_authentication private endpoints.
- Build Private DNS zones and link each to the right VNet, so one URL resolves to different private IPs in different networks.
- Disable public network access and switch to NoAzureDatabricksRules.
- Use for_each to create repeated resources from a single block.
- Verify DNS and connectivity from inside and outside the network.
- Recognise a broken DNS link, where the endpoint looks healthy but traffic never reaches it.

This sets the foundation for hub-and-spoke networking with Azure Firewall, and for connecting on-premises users through VPN or ExpressRoute with DNS forwarding.